



Improving node embedding by a compact neighborhood representation

Ikenna Victor Oluigbo¹ · Hamida Seba¹ · Mohammed Haddad¹

Received: 8 December 2021 / Accepted: 22 November 2022

© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2022

Abstract

Graph Embedding, a learning paradigm that represents graph vertices, edges, and other semantic information about a graph into low-dimensional vectors, has found wide applications in different machine learning tasks. In the past few years, we have had a plethora of methods centered on graph embedding using different techniques, such as spectral classification, matrix factorization and learning. In this context, choosing the appropriate dimension of the obtained embedding remains a fundamental issue. In this paper, we propose a compact representation of a node's neighborhood, including attributes and structure, that can be used as an additional dimension to enrich node embedding, to ensure accuracy. This compact representation ensures that both semantic and structural properties of a node's neighboring-hood are properly captured in a single dimension. Consequently, we improve the learned embedding from state-of-the-art models by introducing the neighborhood compact representation for each node as an additional layer of dimensionality. We leverage on this neighborhood encoding technique and compare with embedding from state-of-the-art models on two learning tasks: node classification and link prediction. The performance evaluation shows that our approach gives a better prediction and classification accuracy in both tasks.

Keywords Node embedding · Embedding dimension · Pairing functions · Node classification · Link prediction

1 Introduction

Graph embedding has gained so much popularity over the years, and it currently lies in the center of many graph analytic problems. Graph embedding is an algorithmic technique which basically transforms graph granularity (nodes, edges, substructures, or whole graph) into low-dimensional continuous vector(s) of predefined number of dimensions. A typical graph (also known as network) has important properties and characteristic features such as topology, structural dependence, node-to-node correlations, neighborhood relationships, etc., which can be

harnessed for a number of learning tasks and analytical problem solving. Therefore, the goal of a graph embedding technique is to ensure that these very valuable graph properties are preserved in the learned embedding. Since graph data can contain millions of vertices and billions of links between the vertices, providing an efficient way (both in time and space complexity) to analyze these massive data is very crucial.

Over the last few decades, several graph embedding models have been designed to learn latent low-dimensional vector(s) from the high dimensionality of graph granular data. A popular way most models learn vector representations from a graph is to first compute an attribute graph (a graph which bears the pairwise similarity feature(s) between these granular data) from the given set of input high dimensional graph granular data. Each granular data having similar attributes are then mapped closely together on a new latent low-dimensional embedding space, and the topological and structural characteristics of the granular data are encoded into its corresponding embedding vector [1]. To obtain low-dimension vectors for graph data,

✉ Hamida Seba
hamida.seba@univ-lyon1.fr

Ikenna Victor Oluigbo
ikenna-victor.oluigbo@etu.univ-lyon1.fr

Mohammed Haddad
mohammed.haddad@univ-lyon1.fr

¹ Univ Lyon, UCBL, CNRS, INSA Lyon, LIRIS, UMR5205, 69622 Villeurbanne, France

several researches have embarked on different approaches ranging from deep neural architecture and spectral processing, to matrix factorization. Many deep architectural approaches make use of an autoencoder. First, an adjacency matrix is computed from the input network, this matrix is then fed into an autoencoder to derive a nonlinear similarity vector representation which preserves the desired information in the embedding space [2]. Instead of computing an adjacency matrix to extract information from an input network (adjacency matrix becomes complex for massive networks), *DeepWalk* [3] and *Node2Vec* [4] use random walks to build a corpus of nodes (made up of a set of fixed length paths, analogous to sentence generation of *Word2Vec*). The SkipGram model is thus applied on the corpus to learn continuous vector representations for nodes by optimizing a neighborhood preserving likelihood objective using Stochastic Gradient Descent.

In the matrix factorization approach, the graph structural properties are first represented as a matrix (node adjacency matrix and Laplacian matrix are popularly used), and then a linear or nonlinear transformation manifold learning procedure is used to factorize the computed matrix (factorization method depends on matrix property) so as to obtain a low-dimensional vector embedding of the network [5, 6].

As much as embedding from popular state-of-the-art representation learning models have shown significant performance in machine learning tasks, it is important to highlight some weaknesses they still possess:

- *Complexity in dimensionality reduction* Many embedding models, especially models needing to compute and factorize complex matrices can be very computationally expensive when transforming data from a high-dimensional space into a low-dimensional space. It becomes even more complex when learning embedding for massive networks while also maintaining high efficiency in preserving important features in the learned embedding. Deep architecture and Random walk approaches enjoy fast training time and low memory consumption but random walks do not consider other features like edge weights and distance, which means that whatever importance a node possesses cannot be preserved in the embedding. Matrix decomposition techniques on the other hand are computationally expensive when factorizing matrix powers.
- *Complexity in information gathering* With the explosive growth of large networks, many existing models are faced with the challenge of capturing and preserving semantic aspects of the network. Some deep architectural models do not consider edge features, meaning that node importance cannot be preserved in the embedding, and such embedding is hard to generalize to new data that are not present when training the model.
- *Estimating embedding dimensionality* In most existing graph embedding methods, the fundamentals involved in correctly appropriating the dimension for an embedding is not well defined. Many methods rely on Grid hyper-parameter estimation or domain experience which can result into issues such as high resource and time usage when training the model, as well as poor model performance [7].
- *Lack of scalability* Many existing representation learning techniques model the low-dimension embedded output as depending wholly on the entire input graph which can be very expensive with massive graphs. As a result of the latency experienced with loading the entire graph into memory, processing can be cut short resulting in intermediate and inconclusive results. This has led to concepts such as embedding compressed subgraph of a massive graph [8, 9].

Also, recent studies have shown that the dimensionality reduction techniques as well as selecting a predefined dimension length for an embedding have an impact on the efficacy of the representation learning model. Selecting a small number of dimensions for learned embedding limits the quality of information captured in the embedding; while a too large dimension increases the possibility of noise and other unnecessary features captured in the embedding which results into a less-than-optimal accuracy in learning tasks, as well as the possible problem of over-fitting a training model for the embedding [7]. A number of literature have proposed techniques for estimating the dimension for embedding. In [10], the authors propose a principled Pairwise Inner Product (PIP) loss technique (adapted from [11]) to choose a dimension value of a generic network embedding. This technique however is limited to capturing only information related to the structural characteristics of nodes in the network. The Pairwise Inner Product (PIP) metric, proposed in [11], uses a grid search algorithm to measure the dissimilarities between word embedding. The grid search technique becomes ineffective when the number of hyper-parameters in the model grows exponentially. In [12], the authors propose a generative network model using an optimal inference algorithm to embed a graph. They achieve this by estimating the probability of nodes pairs captured close in the embedding space and the distance between the node pairs. Their model estimates three embedding dimensions to capture the learned embedding. In [13], the authors propose a metric for k -dimensional structure information embedding of the graph, which is particularly concerned with the structure of the network ignoring its rich link structures. In [14], the authors propose a robust graph dimensionality

reduction algorithm to map high-dimension data into lower-dimensional intrinsic space by a transformation of matrix, which can get very computationally expensive for massive networks. Estimating an appropriate embedding dimension for an efficient and effective embedding in both small and large networks is quite a daunting task for existing embedding models.

In this paper, we attempt to address scalability and dimensionality issues of node embedding. We propose to use a compact encoding of a vertex's neighborhood that combines the structural and semantic features of the neighborhood of the vertex. This encoding process circumvents the technicalities involved with computing complex matrices and appropriating dimensions for learned embedding.

The aim of the compact neighborhood encoding technique is to aggregate all the information surrounding a vertex into a simple numerical value using a bijective function on the node's neighborhood. This representation can be computed without the need to load all the graph in memory eliminating the computational complexity encountered by many existing methods. It also contributes to resolve the challenges involved in estimating the dimension for an embedding. To achieve this objective, we rely on specific kind of bijective functions: *pairing functions* and more precisely the *Cantor polynomial* [15]. These functions have been used to index nodes' neighborhoods for subgraph isomorphism search [16] but to our knowledge there is no work using them to construct node embedding for deep learning purposes. Our main contribution is to use this compact neighborhood encoding to embed in one dimension both structural and semantic information of a node. We evaluate our compact neighborhood representation against selected existing network embedding models on some learning tasks using real world datasets drawn from different domains. This comparison is done across varied dimensions to show the added value of our representation. We show from the experimental results that the proposed compact neighborhood representation technique is efficient and effective for encoding a vertex with vital structural and semantic information around it. This is particularly important for tasks such as predicting new or missing links from/to a vertex.

The remainder of the paper is organized as follows: Sect. 2 presents the definitions and notation used throughout the paper. Section 3 describes related work. In Sect. 4, we present our compact neighborhood representation and its use as an additional feature in node embedding. Section 5 describes the experiments we undertook to evaluate this approach. Section 6 concludes the paper.

2 Preliminaries

In this study, we will use the terms *Network* and *Graph* interchangeably to mean the same thing. Also vertex and node are used interchangeably, as well as edge and link. The vertices are objects in the network, defined by some form of relationship known as links. In this study, we consider node-attributed graphs defined as follows:

Definition 1 We define an attributed Graph G as a 3-tuple $G = (V, E, \Sigma)$, where V consists of the set of vertices in the graph, E consists of the set of edges connecting the vertices in the graph, and Σ is the set of attributes that can appear on the vertices.

Attributes represent the information that the nodes carry and can be of any type. Note that edges can also have attributes.

An undirected graph is a bidirectional graph such that edges (u, v) and (v, u) are equivalent. The neighbors of a vertex u denoted as $\Gamma(u)$ are all vertices adjacent to u . The degree of a vertex u , denoted as $deg(u)$, is the number of neighbors of u .

Figure 1 depicts an example of an undirected node-attributed graph where $V = \{u_1, u_2, u_3, u_4, u_5, u_6, u_7\}$, $E = \{(u_1, u_2), (u_1, u_3), (u_2, u_3), (u_2, u_7), (u_3, u_4), (u_4, u_7), (u_4, u_5), (u_4, u_6)\}$, and $\Sigma = \{A, B, C, D, E\}$.

A graph is commonly represented by its adjacency matrix defined as follows:

Definition 2 An adjacency matrix A of graph $G = (V, E)$, is an $|V| \times |V|$ matrix with non-negative weights, such that $A_{uv} = w_{i,j}$ for a weighted edge or $A_{uv} = 1$ for an unweighted graph if and only if an edge (u, v) exists; else $A_{uv} = 0$ if edge (u, v) does not exists in graph G .

A link prediction model can output whether two nodes are connected by an edge. In a neural link prediction model, the features are aggregated to learn embedding for each node. The similarity score of two node embedding will decide whether they should be connected. Nodes are equally classified based on similarities of their features.

To properly capture the local and global proximity between nodes which are preserved in learned embedding,

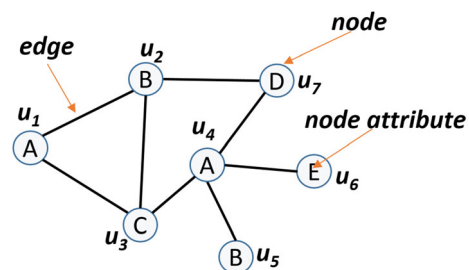


Fig. 1 Example of a node attributed graph.

the main considered similarities are the first-order and second-order proximities [3–5, 17]. The first-order proximity captures the local-pairwise relationship between directly connected vertices in terms of the proximity between vertices in a graph. For a weighted and directed graph having connected vertex pairs $(u, v) \in E$, the first-order proximity is the weight ω_{uv} of the edge between the vertex pair.

With first-order proximity alone many edges in the network goes unobserved, therefore it is not sufficient for preserving the entire structure of the graph.

The second-order proximity defines the neighborhood similarity between two vertices u and v of a directed or undirected network. A second-order proximity exists between u and v if they share a common 1-hop neighbor.

The study, we present here, involves a two prone process; Given an attributed graph $G = (V, E, \Sigma)$, we first compute the compact neighborhood representation for every node in the graph. Our compact neighborhood representation ensures that each node is capable of retaining multiple information from surrounding nodes. Next, we learn embedding with varied dimensions for selected real life datasets using existing network embedding techniques. On the one hand, we experiment with the learned network embedding on two learning tasks (Link Prediction and Node Classification). On the other hand, we add the encoded compact neighborhood representation for each node as an additional feature in the embedding and we repeat the same set of experiments under similar conditions and parameters. After comparison, we focus on determining how much improvement in accuracy and effectiveness we get with the inclusion of the new feature, i.e., the compact neighborhood representation, which has more information embedded in it. The results from the series of test is shown in subsequent sections.

3 Related studies

In this section, we classify some of the existing network embedding models and briefly describe how they perform dimensionality reduction. These models have the same objective of learning embeddings for nodes in a graph, and these embeddings can be used in machine learning tasks.

Random walk approach Embedding models under this approach preserve the structural characteristics of each node or the similarity in node communities, or both [4]. Two main embedding models that leverage on the random walk approach are *Deepwalk* and *Node2Vec*, both of which use the *SkipGram Model* to learn continuous feature representations for words by optimizing a neighborhood preserving likelihood objective (dimensionality reduction). The SkipGram model is a language model (like every other

Word2Vec model) that maximizes the co-occurrence probability among the words that appear within a window in a sentence [18]. To capture information of every node, *Deepwalk*, proposed in [3], takes a graph G and samples uniformly a random vertex u as the root of the random walk with a uniform length. A walk samples uniformly from the neighbors of the last vertex visited until the maximum length for the walk is reached. Upon reaching this maximum length, the walk uniformly samples another random vertex and performs another walk. This walk learns only the first-order proximity of a node, which is not sufficient enough to preserve the entire network structure. *Deepwalk* uses Hierarchical sampling to approximate the softmax probabilities of first-order proximity, and Stochastic Gradient Descent (SGD) to optimize the learning rate and other parameters for feature representation. In contrast, *Node2Vec*, proposed in [4], preserves both the first-order and second-order proximity of a node. For every source node u in the network, *Node2Vec* uses a random walk approach similar to Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms to sample the neighbors of u . In other words, to capture information around a node (Neighbors), *Node2Vec* uses a biased second-order random walk guided by two parameters p and q . Parameter p controls the likelihood of the walk immediately revisiting a node, while parameter q allows the walk differentiate between local view (immediate connected neighbors of a node - BFS) and exploratory view (nodes at farther distance to source node - DFS). The random walk continues until all the nodes in the network are sampled into a large “corpus” of node sequences; the “corpus” is fed into the Skipgram model to learn a mapping of nodes to a low-dimensional space of features that maximizes the likelihood of preserving network neighborhoods of nodes in a δ -dimensional feature space. As with most embedding models, the value of δ is estimated, which can have its impact on the performance of the model. In *BINE* model [19] for bipartite network, the authors capture information around each node into a “corpus” using a probabilistic random walk generator; they then define a joint optimization framework to learn vertex embedding while preserving both explicit and implicit relations simultaneously. In [20, 21], the studies focus on attributed random walks to build sequences of nodes, and then define a conditional probability function that maps nodes with similar attributive relationship in a vector space.

Spectral processing approach There are different variants to this approach; while some architectures have leveraged on spectral heat wavelet diffusion, some other have achieved success through eigen matrix decomposition. For example, an unsupervised model *GRAPHWAVE* proposed in [22] learns a network neighborhood structure into a low-dimensional embedding by leveraging on the

spectral wavelet (unit energy) property of a graph, and the wavelets probability distribution; such that the vertex neighborhood similarity property is preserved in the embedding space. The result from this study was evaluated in areas of node classification, and other evaluation metrics. A Laplacian eigenmaps spectral clustering techniques was proposed in [23] to represent each node into vector embedding by the graph Laplacian eigenvectors associated with its first k nontrivial eigenvalues. The Laplacian Eigenmaps model preserves first-order proximity of nodes in the network and thus, the model computes a larger penalty if two nodes with higher similarity are embedded far apart in the embedding space. Laplacian models only captures the structural properties between node pairs and represents this information in a matrix, it does not capture important node features [24].

Deep architecture approach Most deep architecture models achieves the goal of embedding network properties by means of an encoder. A semi-supervised model *SDNE* [25] was one of the erstwhile deep learning models to learn δ -dimensional embedding with nonlinear functions while preserving the second-order proximity and first-order proximity structure for homogeneous networks. On the other hand, a variational graph autoencoder *VGAE* [26] framework captures node neighborhood information by computing an adjacency matrix A and a degree matrix d , which are entered into a graph convolutional network (GCN) encoder and a simple inner product decoder to learn the higher order dependencies between nodes for tasks such as link prediction. *LINE*, proposed in [17] is a representation learning algorithm that learns an embedding model for real world information networks. To capture the structural features of a network, *LINE* uses both the first-order proximity to preserve the local pairwise proximity between the nodes and second-order proximity to preserve vertices sharing many connections to other vertices. To learn vector embedding for nodes in the network, the *LINE* model is optimized using negative sampling technique to preserve the first-order proximity and second-order proximity separately and then concatenate the embedding trained by the two methods for each vertex. A drawback for this model is that not only it considers the entire network as an input (large networks lead to higher computational complexity), but also it separately trains the objective functions of first-order and second-order proximity resulting into higher time and space complexity.

Matrix factorization approach Nonlinear matrix factorization maps vertices into low-dimensional latent space by considering the eigenvectors of the k -smallest eigenvalues, through nonlinear transformation manifold learning methods, such as Laplacian Eigenmaps (LE), Locally Linear Embedding (LLE), and Isomap. A linear matrix factorization such as Singular Value Decomposition (SVD)

is a complex mathematical technique that factorizes an input adjacency matrix A into the product of three new matrices UDV^T , while ensuring that the three matrices possess some characteristic properties of the original network. U and V are orthonormal $m \times m$ and $n \times n$ real or complex unitary matrix respectively, and D is an $m \times n$ rectangular diagonal matrix with non-negative real numbers on the diagonal. These matrix factorization approaches usually has a high time computational complexity of $\Theta(|V|^2)$, and encounters scalability issues with massive networks.

4 A compact neighborhood representation

As presented in the previous section, embedding graph granularities into low-dimensional continuous vectors can be a complex and computationally expensive task, with high memory utilization. The idea behind compact neighborhood representation (CNR) is to distill in a simple numerical value, i.e., an integer, the neighborhood information surrounding a vertex.

To compute such a representation for a vertex, we use a generalized Cantor Polynomial defined as follows [15]:

$$\begin{aligned} g_k : \mathbb{N}^k &\longrightarrow \mathbb{N} \\ \forall (x_1, x_2, x_3, \dots, x_k) &\in \mathbb{N}^k \text{ and } k > 0 \\ g_k(x_1, x_2, x_3, \dots, x_k) &= \sum_{j=1}^k h(j, x_1 + \dots + x_j) \end{aligned} \quad (1)$$

where

$$h(p, s) = \binom{s+p-1}{p} = \frac{(s+p-1)!}{p!(s-1)!} \quad (2)$$

This is a pairing function known to be a bijective function [27, 28].

To use this bijection on a graph to encode neighborhoods, we first define the one-hop neighborhood of vertex as follows:

Definition 3 The one-hop neighborhood of a vertex v in an attributed graph G is the multiset of attributes appearing on the direct neighbors of v in G . Two vertices u and v have the same one-hop neighborhood in G if they have exactly the same multiset of attributes on their neighbors.

Figure 2 shows an example of two vertices u_1 and u_2 having the same one-hop neighborhood $\{A, B, B, C\}$.

Note that the one-hop neighborhood is defined as a multiset to allow the same attribute to appear multiple times in the same neighborhood. We will also consider that the number of attributes per vertex is one. This means that two vertices that have the same one-hop neighborhood

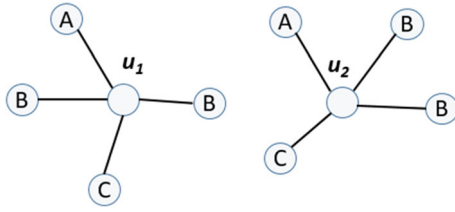


Fig. 2 $\{A, B, B, C\}$ is the One-hop neighborhood of u_1 and u_2 .

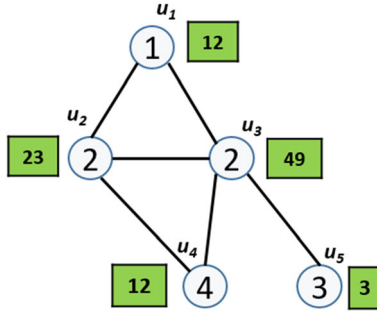


Fig. 3 CNR computation on an example.

have also the same degree. We will see how to deal with this restriction afterward.

Our idea is to assign distinct integer values to distinct vertex attributes. This assignment can be simply achieved by numbering attributes parting from 1. So, the one-hop neighborhood of a vertex will be a multiset of integers. According to this, the value of k , in g_k , is the number of distinct attributes in the graph, i.e., $k = |\Sigma|$ and in the formula $\bar{h}(j, x_1 + \dots + x_j)$, j corresponds to a vertex attribute, and x_j is the number of apparition of attribute j in the direct neighborhood of vertex u .

Example:

Figure 3 illustrates an example where we compute the CNR for 5 vertices, u_1, u_2, u_3, u_4 , and u_5 , on a graph having 4 distinct attributes 1, 2, 3, and 4, i.e., $k = 4$:

$$\begin{aligned} \text{cnr}(u_1) &= g_4(0, 2, 0, 0) = \bar{h}(1, 0) + \bar{h}(2, 0 + 2) \\ &\quad + \bar{h}(3, 0 + 2 + 0) + \bar{h}(4, 0 + 2 + 0 + 0) \\ &= 0 + 3 + 4 + 5 = 12 \end{aligned}$$

. In fact, we can see that attributes 1, 3 and 4 do not appear in the neighborhood of u_1 and attribute 2 appears 2 times. For the remaining vertices, we have similarly:

$$\begin{aligned} \text{cnr}(u_2) &= g_4(1, 1, 0, 1) = \bar{h}(1, 1) + \bar{h}(2, 1 + 1) \\ &\quad + \bar{h}(3, 1 + 1 + 0) \\ &\quad + \bar{h}(4, 1 + 1 + 0 + 1) = 23. \\ \text{cnr}(u_3) &= g_4(1, 1, 1, 1) = \bar{h}(1, 1) + \bar{h}(2, 1 + 1) \\ &\quad + \bar{h}(3, 1 + 1 + 1) \\ &\quad + \bar{h}(4, 1 + 1 + 1 + 1) = 49. \\ \text{cnr}(u_4) &= g_4(0, 2, 0, 0) = \bar{h}(1, 0) + \bar{h}(2, 0 + 2) \\ &\quad + \bar{h}(3, 0 + 2 + 0) \\ &\quad + \bar{h}(4, 0 + 2 + 0 + 0) \\ &= 0 + 3 + 4 + 5 = 12 \\ \text{cnr}(u_5) &= g_4(0, 1, 0, 0) = \bar{h}(1, 0) + \bar{h}(2, 0 + 1) \\ &\quad + \bar{h}(3, 0 + 1 + 0) \\ &\quad + \bar{h}(4, 0 + 1 + 0 + 0) = 3. \end{aligned}$$

We can see that nodes u_1 and u_4 , which have the same 1-hop neighborhood, have the same CNR value.

With this use of the pairing function two vertices in the graph that have not similar one-hop neighborhood cannot have the same CNR even if they have the same degree and the same attribute.

Lemma 1 *Two vertices u and v in a graph G have the same CNR value, i.e., $\text{cnr}(u) = \text{cnr}(v)$, iff u and v have similar one-hop neighborhood.*

Proof We prove the lemma by contradiction. Assume u and v have the same CNR value but their one-hop neighborhood is not alike. Let $a_1, a_2, \dots, a_k$ be the list of the k distinct attributes in G . Without loss of generality, to have different one-hop neighborhoods, it suffices that u has a neighbor with attribute a_j but v has not such a neighbor. This means that the one-hop neighborhoods of u and v differ by one attribute a_j . If we also suppose that all the other attributes are present once in both neighborhoods, this gives:

$$\begin{aligned} \text{cnr}(u) &= g_k(n_{a_1}, n_{a_2}, \dots, n_{a_{j-1}}, 1, n_{a_{j+1}}, \dots, n_{a_k}) \\ \text{cnr}(v) &= g_k(n_{a_1}, n_{a_2}, \dots, n_{a_{j-1}}, 0, n_{a_{j+1}}, \dots, n_{a_k}) \end{aligned}$$

where n_{a_i} denotes the number of occurrences of attribute a_i in the one-neighborhood of the considered vertex. This gives two different sequences of integers that lead to two different CNR values by the bijection of the pairing function which contradicts our hypothesis. The same reasoning holds for the other direction of the equivalence. $\square$

It is worthy to note that the pairing function, and consequently the CNR values, need to compute factorials and will lead to big numbers. To tackle this, we use a simple log function to scale-up the CNR values for each vertex. With this, we effectively handle the “skewness” among the CNR integer outputs without loss of inherent properties

encoded within the integer because log is also a bijective function. It is also interesting to see that the computation of CNR values does not require to load all the graph in memory. Only the direct neighbors of each vertex are needed.

To Compute CNR values for vertices, we supposed till now that each vertex has a single attribute. However, this is not the case in real-world graphs where a vertex may have several attributes. To deal with several attributes per node, we categorize attributes into classes in such a way that nodes sharing similar attributes can be clustered together, and the nodes in the same cluster have the same attribute class value. The class value for each node will be the attribute used in the CNR encoding technique. To process the sequence of binary and discrete/continuous node attributes into attribute classes, we follow a series of steps.

In the first step, we used the Hamming distance algorithm to measure the similarity or dissimilarity between each node attributes sequences. Using this distance measure, we can cluster nodes together; however, we need to know the optimal number of clusters k . To do this, we use the Elbow method, Silhouette score, and Dendrogram plot. The Elbow method and Dendrogram plot gives probabilities for different possible number of clusters, while the Silhouette score gives a coefficient value for each cluster. Naturally, we chose the k having the best coefficient value which happens to have the highest probability also. Basically, we want to ensure that we are able to compare different methods for the correct k value. As a final step, we apply the kMeans clustering technique using the k clusters to predict the cluster to which each node belongs to. Each node is assigned an attribute class value, corresponding to the cluster it belongs in. The attribute class value for each node is used for its CNR encoding, while k corresponds to the unique number of attributes for the CNR bijection function.

5 Experimental evaluation

In this comparative study, we evaluate how the learned embedding from three state-of-the-art embedding models *DeepWalk* [3], *Node2vec* [4], and *LINE* [17] perform on *Link Prediction and Node Classification tasks*; first as stand-alone embedding, and then with the introduction of CNR encoding as an additional important feature. For each dataset used in this study, we learned embedding in 32, 64, and 128 dimensions. The aim is to find out how this additional feature affects the overall result of the tasks. Specifically, we focus on the following research question: To what extent does introducing an additional encoded feature (i.e., CNR) impacts on the overall performance in the link prediction and node classification tasks?

5.1 Datasets

In our experiments, we used seven real-life datasets (see Table 1 for a summary) from different domains [29]:

- The Cora dataset is a citation network, where nodes are publications and edges are citation links. Attributes of each node are categorical binary representations of the corresponding publications.
- The FIRSTMM DB dataset contains a set of graphs corresponding to 3D point cloud data and categories of various household objects for semantic and graph-based object category prediction and has 33 training graphs and 4 graphs each for validation and testing. Attributes of each node are discrete/continuous representations.
- The proteins dataset is a collection of proteins that are classified as enzymes or non-enzymes. Nodes represent the amino acids and two nodes are connected by an edge if they are less than 6 Angstroms apart. Attributes of each node are discrete/continuous representations.
- The Enzymes dataset consists of 600 protein tertiary structures obtained from the BRENDA enzyme database. The enzymes dataset contains 6 enzymes, corresponding to its attributes. Attributes of each node are discrete/continuous representations
- The Dihydrofolate Reductase Inhibitors (DHFR) dataset contains values for 325 compounds. For each compound, 228 molecular descriptors have been calculated. Additionally, each sample is designated as “active” or “inactive”. Attributes of each node are discrete/continuous representations.
- The Soc-political-retweet networks consists of the most retweeted user in a political debate. Nodes are the users of the twitter social network while the edges represents the retweets between them. This dataset have no attributes for nodes.
- The NCI-1 dataset consists of small molecules with class labels representing toxicity or biological activity determined in drug discovery projects. Here, the nodes take the places of atoms and edges are the chemical bonds between atoms. This dataset have no attributes for nodes.

Note that the two last datasets have no node attributes. This allow us to test the CNR approach on such networks. To apply CNR on datasets without node attributes, we used the adjacency matrix to retrieve the structural relationship between nodes in the network. The rows in the matrix are such that 1 denotes if two nodes are connected and 0 if they are not connected. This way, we can construct a categorical binary sequence of 1's and 0's representing the attributes for each node in the network. We then apply the same process described in the previous section to assign an

Table 1 Datasets

Benchmark Datasets	#Nodes	#Edges	Density	Maximum Degree	#Class Labels
Cora	2708	5429	$1.4 * 10^{-3}$	169	7
FIRSTMM	56.6 k	252 k	$1.5 * 10^{-4}$	20	5
Proteins	43.5 k	162.1 k	$1.7 * 10^{-4}$	50	3
NCI-1	122.3 k	265.5 k	$3.5 * 10^{-5}$	8	25
Enzymes	19.5 k	75 k	$3.9 * 10^{-4}$	18	3
DHFR	32 k	67 k	$1.3 * 10^{-4}$	8	9
Political Retweets	18.5 k	61.2 k	$3.5 * 10^{-4}$	1k	2

attribute class to each node. Note that, in this case, the CNR contains only structural information.

5.2 Methods and settings

As already stated, we adopted the Link Prediction and Node Classification tasks to answer the question posed previously in this section. For the purpose of achieving the link prediction analysis, we designed a logistic regression model for labeled indices. We consciously opted for such model because (i) It is very efficient to train, and fast at classifying unknown occurrences. (ii) Since our features are linearly separable with no multicollinearity, this model is ideal. (iii) It is effective at interpreting model coefficients based on important attributes in the features. (iv) It is less inclined to overfitting. The idea behind the link prediction task is to measure the accuracy at predicting missing links in the network. To achieve this, we consider a network with certain fractions of edges removed. We generated the training dataset by randomly removing varying percentage of edges from the network while also ensuring that the network obtained after removing the edges still remains connected. For the node classification task however, we designed a Graph Convolutional Network (GCN) model. The GCN hyperparameters includes 2 layers with hidden size 16, training was done using the Adam Optimizer [30] and softmax was used for output classification, with learning rate 0.01 and weight decay 5×10^{-4} .

To measure the models' performance, we use the following metrics:

- Area under the precision-recall curve: Recall measures what percentage of positives were returned. Precision measures what percentage of the results are true positives. These metrics are used to construct a Precision-Recall Curve which shows how the increase in recall affects precision.
- Area under the receiver operating characteristics curve: A combination of the True positive rate (recall) and the false positive rate which measures how many negatives

were returned as false positives, are used to construct a Receiver Operating Characteristics (ROC).

- Precision@k: The Precision at k metric returns the percentage of true positives among only the top k ranked links. It is a metric for measuring link equality. Mathematically, precision is expressed as
$$\frac{\text{True Positive}(TP)}{\text{True Positive}(TP) + \text{False Positive}(FP)}.$$
- F-score: The F-score (also known as F1-score) is a measure of a model accuracy on a dataset. The F-score is a way of combining the precision and recall of the model, and it is defined as the harmonic mean of the model precision and recall. Mathematically, it is represented as
$$2 \times \left(\frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \right).$$

5.3 Results for link prediction experiments

For this task, we tried to evaluate how well our model predicted missing links using the information embedded in the learned representation. In order to get a baseline score for link prediction, we selected five heuristics metrics:

- *Common Neighbor*: $CN = |\Gamma(x) \cap \Gamma(y)|$,
- *Jaccard's Coefficient* = $|\frac{\Gamma(x) \cap \Gamma(y)}{\Gamma(x) \cup \Gamma(y)}|$,
- *Preferential Attachment*: $PA = |\Gamma(x)| |\Gamma(y)|$,
- *Adamic/Adar Index*: $AI = \sum_{z \in \Gamma(x) \cap \Gamma(y)} \frac{1}{\log |\Gamma(z)|}$, and
- *Resource Allocation Index* = $\sum_{z \in \Gamma(x) \cap \Gamma(y)} \frac{1}{|\Gamma(z)|}$.

where x and y denotes nodes, $\Gamma(x)$ and $\Gamma(y)$ denotes the neighbor set of these nodes, while z is a common neighbor of node x and node y [31]. Scores of heuristics for every dataset are given in Table 2.

To evaluate performance of learning models, all nodes and edges were taken into consideration; and we ensured that while splitting our data into training and test data, the network remained largely connected thus avoiding disconnected components. The dataset was split in the ratio 80:20 with 80% of the data used for training the model and the remaining 20% for evaluation. We first conducted a link prediction analysis using varied dimensions (32, 64,

Table 2 Link prediction heuristic scores for the seven selected networks

Heuristic method	Cora	First MM	Protein	Enzymes	Political Retweets	DHFR	NCI 1
Common neighbor	0.6631	0.6843	0.7119	0.7481	0.7012	0.7567	0.7579
Jaccard's neighbor	0.6580	0.6729	0.6932	0.6610	0.6521	0.6341	0.6737
Preferential attachment	0.6873	0.6891	0.7141	0.6783	0.7012	0.6862	0.7119
Adamic/adar index	0.7063	0.7168	0.7585	0.7705	0.7164	0.7521	0.7781
Resource allocation	0.6831	0.7045	0.6951	0.6859	0.7067	0.6872	0.7106

and 128) of learned embedding from the three selected state-of-the-art models. For the second part, we merge the originally learned embedding with the encoded CNR feature for each node and we perform the same series of link prediction analysis a second time using the newly merged features and under the same set of conditions and parameters. The results for link prediction analysis is discussed in this section.

We discuss the results of the link prediction analysis from the perspective of each of the embedding. The results in Table 3 pretty much shows the effect of the CNR encoding as an additional feature in the learned embedding.

While we got a decent prediction accuracy using learned embedding from state-of-the-art models, clearly the introduction of an extra encoded feature for each vertex increased the accuracy of our link prediction model in predicting previously unseen connections considerably. This is possible due to a number of reasons which includes:

- (1) CNR encoding is sensitive to structural features of the network, since it encodes node based on other nodes in its neighborhood. And since the link prediction analysis involves predicting structural connection between nodes, it makes the introduction of CNR improve the predictive accuracy of the model.
- (2) For each vertex u , the information stored in the surrounding neighbors of u are encoded as a single numerical value for vertex u . Each neighbor of u also takes and encodes information about its respective neighbors. As a result of this, each node has the capacity of storing several metadata in a single numerical value. This property gives graphs encoded with CNR technique the ability to preserve an entire graph property in a set of numerical values.

In addition, across the AUC results for all seven datasets, our model was able to correctly predict the true positive edges out of all the edges removed from the dataset for testing. This is reflected in the high Precision values, which grows even higher when we factored in the CNR encoding. Another interesting finding is that for every increment in the embedding dimensions, we record a lower link prediction accuracy which reflects for AUCs without CNR and with the introduction of CNR encoded values. This could

be as a result of adding several unnecessary features which reduces the performance of the embeddings in learning tasks.

Finally when we compare our results with the standard heuristics scores shown in Table 2, we get a much higher differentials for the link prediction analysis with CNR. From our findings, it seems clear that CNR encoding thrives well as an additional dimension for node embedding in networks. And even for massive networks with higher maximum degree where each node in the network have more surrounding nodes, there are more surrounding information in each node neighborhood to encode, adding to the flexibility of CNR. As a result of this unique property of CNR, its application can cut across diverse domains as well as for different learning tasks.

5.4 Results for node classification experiments

In many real-life networks, the nodes in the network can have some kind of relationships based on certain intrinsic characteristics of each node. For example, we might want to classify nodes based on the number of inward/outward edges, or classify nodes on the same structural neighborhood, or sometimes find some unknown relationships in an unsupervised environment. In our study, we first learn vector embedding for nodes in an attributed graph, and then we designed a GCN model to classify the nodes using the learned embedding. The model was trained with 200 epochs, early stopping on validation loss with 10 epochs of patience, and a categorical cross-entropy loss function. On the other hand, we conduct the same experiment under the same environment and parameters but this time, with CNR encoding for each node added as an additional feature in the embedding. As a way of quality and accuracy assurance, we repeat both sets of experiments with a 32, 64, and 128 dimension embedding. We evaluate the results from our experiments using the F1-score [32, 33] to measure how precise the model accuracy was for the node classification experiment.

The results in Fig. 4 and F1-score in Table 4 show significant degree of improvement in the classification accuracy with the additional CNR as an extra embedding layer for each node. The precision@ k is used to measure the confidence level of the model's classification result. In

Table 3 Evaluation results : accuracy precision recall

Dataset dimension	Node2Vec	Node2vec Plus CNR	DeepWalk	DeepWalk Plus CNR	LINE	LINE Plus CNR
Cora32	0.8703 0.8860 0.8703	0.8572 0.8631 0.8764	0.8562 0.8797 0.8526	0.8439 0.8436 0.8439	0.7512 0.7720 0.7512	0.7667 0.7510 0.7667
Cora64	0.7658 0.7905 0.7658	0.8430 0.7561 0.8422	0.7529 0.7941 0.7529	0.8681 0.8179 0.8681	0.5407 0.5750 0.5407	0.7617 0.7605 0.7612
Cora128	0.7892 0.7180 0.7892	0.8457 0.8385 0.8457	0.6525 0.6527 0.6525	0.8511 0.8176 0.8559	0.5395 0.5688 0.5395	0.7564 0.7553 0.7564
Proteins32	0.8861 0.9132 0.8929	0.8898 0.8752 0.8878	0.8913 0.9262 0.8923	0.8948 0.8739 0.8914	0.7921 0.7666 0.7921	0.8815 0.9247 0.8815
Proteins64	0.7378 0.7518 0.7378	0.8779 0.8431 0.8779	0.7736 0.7979 0.7736	0.8863 0.8561 0.8863	0.7103 0.7047 0.7103	0.8790 0.9172 0.8836
Proteins128	0.7364 0.7295 0.7364	0.8665 0.8617 0.8655	0.7501 0.7975 0.7501	0.8951 0.9020 0.8956	0.6708 0.6857 0.6708	0.8834 0.9287 0.8834
FirstMM32	0.9275 0.9353 0.9268	0.9881 0.9398 0.9881	0.8436 0.8582 0.8436	0.9706 0.9751 0.9711	0.7671 0.7993 0.7671	0.9738 0.9871 0.9443
FirstMM64	0.8419 0.8487 0.8341	0.9870 0.9446 0.9891	0.7464 0.7629 0.7464	0.9861 0.9756 0.9861	0.7533 0.7989 0.7533	0.9782 0.9336 0.9748
FirstMM128	0.8652 0.8678 0.8606	0.9671 0.9647 0.9552	0.7871 0.8360 0.7839	0.9862 0.9837 0.9744	0.7310 0.7732 0.7310	0.9225 0.9237 0.9225
Enzymes32	0.7904 0.7818 0.7815	0.9425 0.9136 0.9425	0.7676 0.7692 0.7687	0.9350 0.9491 0.9350	0.7844 0.7739 0.7527	0.8668 0.8524 0.8668
Enzymes64	0.7419 0.7233 0.7384	0.8961 0.8847 0.8961	0.7248 0.7176 0.7145	0.8649 0.8430 0.8649	0.7403 0.7601 0.7419	0.8320 0.8487 0.8320
Enzymes128	0.7257 0.7122 0.7241	0.8451 0.8539 0.8451	0.7104 0.6615 0.6973	0.8446 0.8520 0.8446	0.7485 0.7322 0.7477	0.8334 0.8350 0.8361
DHFR32	0.7631 0.7642 0.7681	0.9266 0.9091 0.9264	0.7569 0.7412 0.7510	0.8751 0.8313 0.8730	0.7422 0.7448 0.7369	0.8854 0.8039 0.8617
DHFR64	0.7730 0.7793 0.7750	0.9051 0.8874 0.9051	0.7487 0.7453 0.7490	0.8564 0.8325 0.8564	0.7124 0.7022 0.7124	0.8751 0.8335 0.8735
DHFR128	0.7311 0.7328 0.7239	0.8944 0.8439 0.8944	0.7178 0.7106 0.7131	0.8088 0.8324 0.8257	0.6889 0.6919 0.6910	0.8237 0.8091 0.8237
Politics32	0.8852 0.8743 0.8809	0.9847 0.9821 0.9864	0.8698 0.8703 0.8698	0.9683 0.9691 0.9683	0.7951 0.7903 0.7947	0.9165 0.9114 0.9148
Politics64	0.8365 0.8420 0.8365	0.9627 0.9501 0.9627	0.8665 0.8612 0.8665	0.9144 0.9062 0.9144	0.7538 0.7491 0.7557	0.8691 0.8736 0.8674
Politics128	0.7976 0.7933 0.7905	0.8778 0.8781 0.8778	0.7743 0.7718 0.7743	0.8971 0.8746 0.8971	0.7385 0.7485 0.7393	0.8577 0.8772 0.8591
NCI32	0.8852 0.8736 0.8835	0.9770 0.9795 0.9862	0.8733 0.8609 0.8739	0.9433 0.9257 0.9433	0.7752 0.7640 0.7721	0.8768 0.8349 0.8724
NCI64	0.8305 0.8365 0.8305	0.9154 0.9263 0.9154	0.8115 0.8296 0.8115	0.8944 0.8863 0.8940	0.7665 0.7513 0.7668	0.8413 0.8470 0.8446
NCI128	0.7622 0.7706 0.7622	0.8735 0.8411 0.8810	0.7733 0.7619 0.7733	0.8351 0.8492 0.8351	0.7361 0.7460 0.7361	0.8115 0.8078 0.8153

simple terms, Precision is the ratio between the True Positives (correctly predicted positive observations) and all the Positives (total predicted positive observations) in the dataset. Therefore a high precision means a low false positive rate, and vice versa. In the context of our study, we use precision to measure nodes that were correctly classified into their respective class out of all the nodes in each

class. For example, if a class has 50 nodes, a precision of 0.70 means the model correctly classified about 35 nodes as belonging to that class out of the total 50 nodes. For each dataset, we run the classification model for 32, 64, and 128 dimension embedding for *Node2Vec*, *DeepWalk*, and *LINE* embedding models. While we achieve a decent classification accuracy with *Node2Vec* and *Deepwalk* models, the



Fig. 4 Precision@ k Results for Node Classification (DW and N2V denotes DeepWalk and Node2Vec, respectively). Orange bar = Embedding plus CNR, Blue bar = Embedding without CNR

addition of the compact neighborhood representation significantly improves the classification accuracy as seen in the F1-score. With the addition of CNR as an extra layer of

embedding the classification model accurately classified the nodes by over 80% in almost all the datasets having node attributes. The F1-score takes both false positives and

Table 4 F1-Score

Dataset dimension	Node2Vec	Node2vec Plus CNR	DeepWalk	DeepWalk Plus CNR	LINE	LINE Plus CNR
Cora32	0.65	0.87	0.66	0.76	0.62	0.69
Cora64	0.64	0.83	0.60	0.82	0.52	0.65
Cora128	0.60	0.83	0.58	0.80	0.50	0.69
Proteins32	0.68	0.75	0.69	0.72	0.66	0.69
Proteins64	0.66	0.73	0.64	0.72	0.61	0.64
Proteins128	0.58	0.70	0.60	0.68	0.60	0.60
FirstMM32	0.67	0.74	0.65	0.73	0.63	0.71
FirstMM64	0.65	0.72	0.62	0.71	0.64	0.71
FirstMM128	0.68	0.70	0.62	0.71	0.62	0.65
Enzymes32	0.72	0.88	0.69	0.84	0.69	0.82
Enzymes64	0.70	0.84	0.70	0.81	0.68	0.77
Enzymes128	0.65	0.80	0.66	0.81	0.68	0.77
DHFR32	0.78	0.88	0.76	0.85	0.73	0.82
DHFR64	0.77	0.85	0.71	0.83	0.71	0.82
DHFR128	0.66	0.79	0.64	0.79	0.64	0.75
Politics32	0.79	0.77	0.77	0.75	0.72	0.72
Politics64	0.75	0.74	0.76	0.77	0.69	0.71
Politics128	0.78	0.76	0.74	0.75	0.70	0.70
NCI32	0.75	0.75	0.77	0.80	0.71	0.74
NCI64	0.73	0.74	0.73	0.72	0.70	0.72
NCI128	0.72	0.71	0.72	0.71	0.71	0.72

*Dim denotes 32/64/128 Dimensions varied for each dataset

false negatives into account, with a good F1-score indicative of a good Precision accuracy. The result in Table 4 correlates with the precision accuracy in Fig. 4, with the F1-score considerably higher with CNR encoding. For the two datasets without node attributes, there was not much improvement in the classification result. This could be as a result of the fact that the features possessed by each node plays a huge role in properly classifying the nodes into their correct category.

However, we still obtained good classification score with CNR as an extra layer of embedding. As with the case of the Link prediction analysis, it is evident that as we vary the number of dimensions from 32 through 128, the classification accuracy reduces; the *curse of dimensionality theory* quickly springs to mind. This theory indicates that as the number of dimensions increases, so does the addition of repetitive and sometimes unnecessary features (perhaps noise), which increases the model error and thus results in a loss in accuracy.

It is therefore important to determine an optimal embedding length in a representation learning problem, so as to get a good embedding output for machine learning tasks. Compact Neighborhood Representation can be an

interesting solution to this problem since we are able to encode every information and characteristic feature surrounding a node in just a single numerical value.

In Fig. 5, we try to visualize the hidden layer representations of our model in 2-dimensions. Each point on the plane represents each node in the network, while each color represents each class to which a node belongs in. Figure 5 shows the hidden layer activation for Cora dataset which consists of machine learning papers grouped in the following seven label; Case Based, Genetic Algorithms, Neural Networks, Probabilistic Methods, Reinforcement Learning, Rule Learning, and Theory. Compared to the visualization in Fig. 5a without CNR, Fig. 5b shows some degree of class distinction for the seven classes of papers in the dataset. We still see some papers in other clusters, this can be attributed to the fact that in the document corpus, some papers were cited more than once. However, we can still get a clear picture of the different classes of papers in Fig. 5b compared to Fig. 5a. Overall, the node classification analysis with CNR encoding outperforms the classification without CNR by up to 15%, which is a significant improvement in accuracy. As described in Sect. 4, since each vertex in a network is capable of effectively holding higher-order information about other vertices (described in

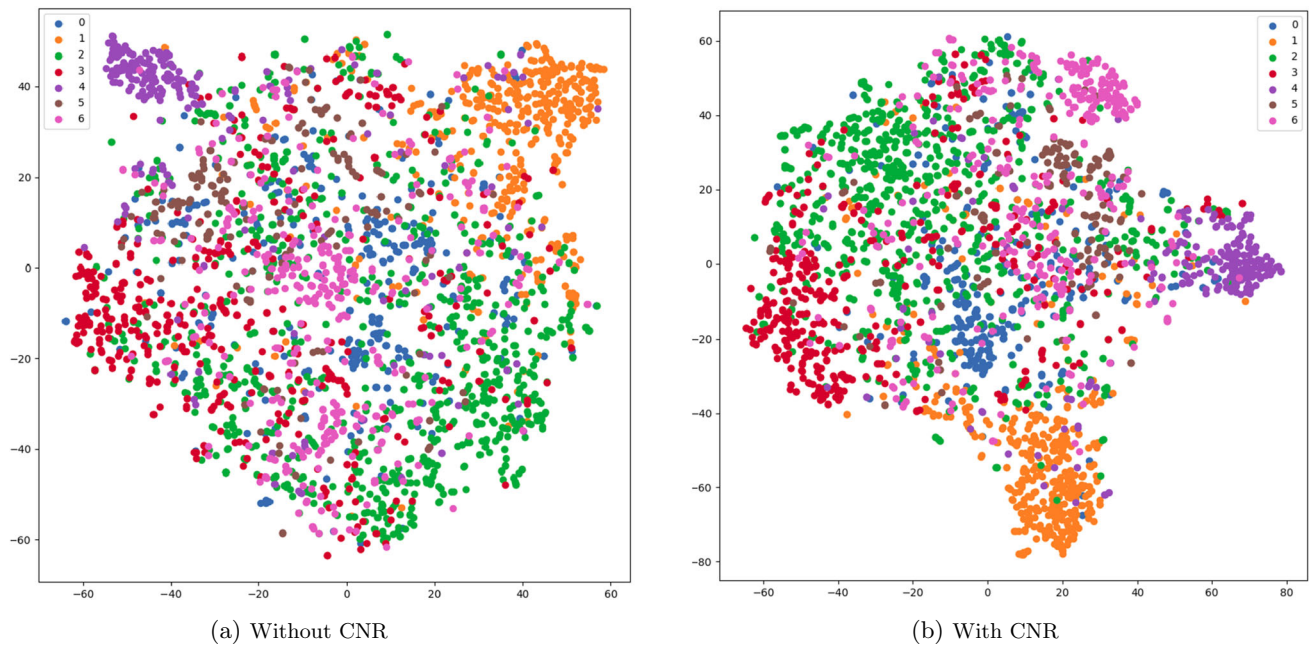


Fig. 5 Visual display for Cora Dataset with 7 categories

the node attributes), the neighborhood compact representation technique is quite effective for node classification.

6 Conclusion

This study involves a performance evaluation of state-of-the-art embedding models (Node2Vec [4], DeepWalk [3], and LINE [17]) against a compact neighborhood representation technique that captures in a single dimension both structural and semantic information around vertices. We conducted two machine learning tasks on seven real life networks, with learned vector representations from those models on one hand, and then we introduced the CNR encoding as an additional feature for an improved accuracy on the other hand. Results from both experiments show that we were able to improve our link prediction and node classification accuracy significantly with the addition of the CNR feature as an additional layer of embedding. This confirms that this vertex representation carries important information about nodes. Additionally, we get a better result with CNR as an additional layer of embedding over the features captured by existing embedding methods. As part of ongoing work, we will like to explore how much role CNR encoding will play in massive temporal and streaming networks in areas of detecting anomalies and pattern matching. Another interesting area is to represent each distinct characteristic feature in a network with a CNR, thus being able to effectively distinguish a feature from another and perform certain tasks on them separately. It is also interesting to see that CNR can be easily extended

to encode edge attributes and k -hop neighboring-hood. This may enhance the accuracy of the computed embedding in certain applications.

Acknowledgements This work was supported by the Agence National de la Recherche (ANR) under Grant Agreement No ANR-20-CE23-0002, and Petroleum Technology Development Fund, Nigeria with grant number PTDF/GFC/035.

Author Contributions The study conception and design was performed by HS. Implementation and analyses were performed by IO and HH. The first draft of the manuscript was written by IO and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding For the research leading to these results, Hamida Seba and Mohammed Haddad received funding from Agence National de la Recherche (ANR) under Grant Agreement No ANR-20-CE23-0002, Ikenna Oluigbo was supported by Petroleum Technology Development Fund, Nigeria with grant number PTDF/GFC/035,

Declarations

Conflict of interest All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript.

Ethics approval All authors declare that they adhere to the ethical principles of the journal

Code availability The source code of the algorithms and the related data are available in <https://gitlab.liris.cnrs.fr/hseba/cnr>.

References

- Narayanan A, Chandramohan M, Venkatesan R, Chen L, Liu YP, Jaiswal S (2017) graph2vec: Learning distributed representations of graphs. [arXiv: abs/1707.05005](#)
- Tu K, Cui P, Wang X, Wang F, Zhu W (2017) Structural deep embedding for hyper-networks. [arXiv preprint arXiv:1711.10146](#)
- Perozzi B, Al-Rfou R, Skiena S (2014) Deepwalk: Online learning of social representations. In: Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data mining, pp 701–710. ACM
- Grover A, Leskovec J (2016) node2vec: Scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 855–864. ACM
- Cao S, Lu W, Xu Q (2015) Grarep: Learning graph representations with global structural information. In: Proceedings of the 24th ACM international on conference on information and knowledge management CIKM '15, pp 891–900. ACM. <https://doi.org/10.1145/2806416.2806512>
- Perozzi B, Kulkarni V, Chen H, Skiena S (2017) Don't walk, skip!: Online learning of multi-scale network embeddings. In: Proceedings of the 2017 IEEE/ACM international conference on advances in social networks analysis and mining ASONAM '17, pp 258–265. ACM. <https://doi.org/10.1145/3110025.3110086>
- Luo G-X, Li J, Peng H, Yang C, Sun L, Yu P-S, He L (2021) Graph entropy guided node embedding dimension selection for graph neural networks. In: IJCAI
- Adhikari B, Zhang Y, Ramakrishnan N, Prakash B.A (2018) Sub2vec: Feature learning for subgraphs. In: PAKDD
- Balalau O, Goyal S (2020) Subrank: Subgraph embeddings via a subgraph proximity measure. *Adv Knowl Discovery Data Mining* 12084:487–498
- Gu W, Tandon A, Ahn Y-Y, Radicchi F (2021) Principled approach to the selection of the embedding dimension of networks. *Nature Commun.* <https://doi.org/10.1038/s41467-021-23795-5>
- Yin Z, Shen Y (2018) On the dimensionality of word embedding. In: NeurIPS
- Hoff PD, Raftery AE, Handcock MS (2002) Latent space approaches to social network analysis. *J Am Stat Assoc* 97:1090–1098
- Li A, Pan Y (2016) Structural information and dynamical complexity of networks. *IEEE Trans Inf Theory* 62:3290–3339
- Zhu X, Lei C, Yu H, Li Y, Gan J, Zhang S (2018) Robust graph dimensionality reduction. In: Proceedings of the 27th international joint conference on artificial intelligence, pp 3257–3263. AAAI Press, ???
- Rudolf Fueter GP (1923) Rationale abzählung der gitterpunkte, vierteljschr. Naturforsch. Ges, Zürich 58, 280–386
- Nabti C, Seba H (2017) Compact neighborhood index for subgraph queries in massive graphs. [arXiv: Databases](#)
- Jian T, Meng Q, Mingzhe W, Ming Z, Jun Y, Qiaozhu M (2015) Line: Large-scale information network embedding. In: Proceedings of the 24th international conference on world wide web, pp 1067–1077
- Mikolov T, Chen K, Corrado G.S, Dean J (2013) Efficient estimation of word representations in vector space. *Comput Res Repository (CoRR)*. [arXiv:1301.3781](#)
- Gao M, Chen L, He X, Zhou, A (2018) Bine: Bipartite network embedding. In: Proceedings of the 41st international ACM SIGIR conference on research and development in information retrieval SIGIR '18, pp 715–724. ACM. <https://doi.org/10.1145/3209978.3209987>
- Ahmed N.K, Rossi R, Lee J.B, Willke T.L, Zhou R, Kong X, Eldardiry H (2018) Learning role-based graph embedding, pp 1–8 [arXiv:1802.02896v2](#)
- Sheikh N, Kefato ZT, Montresor A (2018) gat2vec: representation learning for attributed graphs. *J Comput* 101(3):187–209
- Donnat C, Zitnik M, Hallac D, Leskovec J (2018) Learning structural node embeddings via diffusion wavelets. In: Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery and data mining, pp 1320–1329. ACM
- Belkin M, Niyogi P (2002) Laplacian eigenmaps and spectral techniques for embedding and clustering. In: NIPS
- Makarov I, Kiselev D, Nikitinsky N, Subelj L (2021) Survey on graph embeddings and their applications to machine learning problems on graphs. *PeerJ Comput Sci.* 7
- Wang D, Cui P, Zhu W (2016) Structural deep network embedding. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining KDD '16, pp 1225–1234. ACM. <https://doi.org/10.1145/2939672.2939753>
- Kipf T, Welling M (2016) Variational graph auto-encoders. In: NIPS workshop on Bayesian deep learning [arXiv: 1611.07308](#)
- Nathanson M (2015) Cantor polynomials and the Fueter–Pólya theorem. *Am Math Monthly* 123:1001–1012
- Lisi M (2007) Some remarks on the cantor pairing function. *LE MATEMATICHE LXII-Fasc. I*, pp 55–65
- Ryan R, Nesreen A (2015) The network data repository with interactive graph analytics and visualization. In: AAAI. <http://networkrepository.com>
- Diederik K, Jimmy B (2015) Adam: A method for stochastic optimization. *CoRR* [arXiv: 1412.6980](#)
- Gao F, Musial K, Cooper C, Tsoka S (2015) Link prediction methods and their accuracy for different social networks and network metrics. *Sci Program* 2015:172879–117287913
- Yacouby R, Axman D (2020) Probabilistic extension of precision, recall, and f1 score for more thorough evaluation of classification models. In: EVAL4NLP
- Lan L, Wang P, Du X, Song K, Tao J, Guan X (2020) Node classification on graphs with few-shot novel labels via meta transformed network embedding. [arXiv: abs/2007.02914](#)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.